

Bilkent University, CS 473/573 - Algorithms I, Midterm Exam Cheat Sheet

Write ONLY your name on this paper, do NOT write anything else.

Substitution Method

1. Guess the asymptotic complexity.
2. Prove your guess using induction
 - (a) Assume inductive hypothesis holds for $k < n$
 - (b) Try to prove the general case for n

Note: You MUST prove the EXACT inequality, you CANNOT ignore lower order terms.
If the proof fails, strengthen the inductive hypothesis and try again.

Master Method:

Let $a \geq 1, b > 1$ be constants, let $f(n)$ be an asymptotically positive function, and let $T(n)$ be defined on the nonnegative integers by the recurrence; for

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

- Case 1: $\left\{ \frac{n^{\log_b a}}{f(n)} = \Omega(n^\varepsilon) \ \varepsilon > 0 \right\} \Rightarrow T(n) = \Theta(n^{\log_b a})$
- Case 2: $\left\{ \frac{f(n)}{n^{\log_b a}} = \Theta(lg^k n) \right\} \Rightarrow T(n) = \Theta(n^{\log_b a} lg^{k+1} n)$
- Case 3: $\left\{ \frac{f(n)}{n^{\log_b a}} = \Omega(n^\varepsilon) \ \text{and} \ af(n/b) \leq cf(n) \ \text{for} \ c < 1 \ \text{and} \ \varepsilon > 0 \right\} \Rightarrow T(n) = \Theta(f(n))$

You can use the following algorithms covered in the lectures/book as indicated below:

- **INSERTION-SORT**(A, n): Insertion sort on $A[1 \dots n]$
- **MERGE-SORT**(A, p, r): Merge sort on $A[p \dots r]$
- **BINARY-SEARCH**(A, p, r, key): Binary search for key on $A[p \dots r]$
- **H-PARTITION**(A, p, r): Hoare's partitioning on $A[p \dots r]$, returns the last position of the region $\leq pivot$
- **L-PARTITION**(A, p, r): Lomuto's partitioning on $A[p \dots r]$, returns the last position of the region $\leq pivot$
- **QUICKSORT**(A, p, r): Quicksort on $A[p \dots r]$
- **R-QUICKSORT**(A, p, r): Randomized quicksort on $A[p \dots r]$
- **R-H-PARTITION**(A, p, r): Randomized Hoare's partitioning on $A[p \dots r]$ with a randomly selected pivot, returns the last position of the region $\leq pivot$
- **R-L-PARTITION**(A, p, r): Randomized Lomuto's partitioning on $A[p \dots r]$ with a randomly selected pivot, returns the last position of the region $\leq pivot$
- **R-SELECT**(A, p, r, i): Randomized select algorithm on array $A[p \dots r]$, returns the value of the i th smallest element
- **MM-SELECT**(S, n, i): Median of medians algorithm on set S containing n elements, returns the value of the i th smallest element
- **HEAP-EXTRACT-MAX**(A, n): Extract max on the heap of n elements stored in $A[1 \dots n]$
- **MAX-HEAPIFY**(A, i, n): Max heapify on the i th element of the heap stored in $A[1 \dots n]$
- **BUILD-MAX-HEAP**(A, n): Builds heap in $A[1 \dots n]$ using the n elements given in $A[1 \dots n]$
- **HEAPSORT**(A, n): Heapsort on $A[1 \dots n]$
- **MAX-HEAP-INSERT**(A, key, n): Inserts key into the heap stored in $A[1 \dots n]$
- **HEAP-INCREASE-KEY**(A, i, key): Increases the value of the i th element of the heap to key
- **COUNTING-SORT**(A, B, n, k): Counting sort on $A[1 \dots n]$ with integers in the range $\{1, 2, \dots, k\}$, returns the sorted array $B[1 \dots n]$